Dance Genre Classification

Link: https://github.com/kunathmi/cmse492_project.git

This project builds a classification algorithm that can classify keypoint data of body movements in dance videos into two different dance styles. Specifically, I used keypoint data which identifies major joints and key facial features and tracks their locations in space throughout a video. Then, I built a feature vector for each dance sample using various calculations on the keypoint data. These calculations were determined in order to identify a dance such as velocity or "energy," range of motion, etc. I then used the feature vectors to build a machine learning model to classify dance videos into two different styles - Jazz Ballet or House. Without machine learning, this project could really only be solved by having a dance expert sit down and watch hundreds of videos and classify them into each genre. This would take a lot of time and resources. Also, there may be human error, especially if the expert gets tired. Machine learning is a better approach because it is much faster and less expensive.

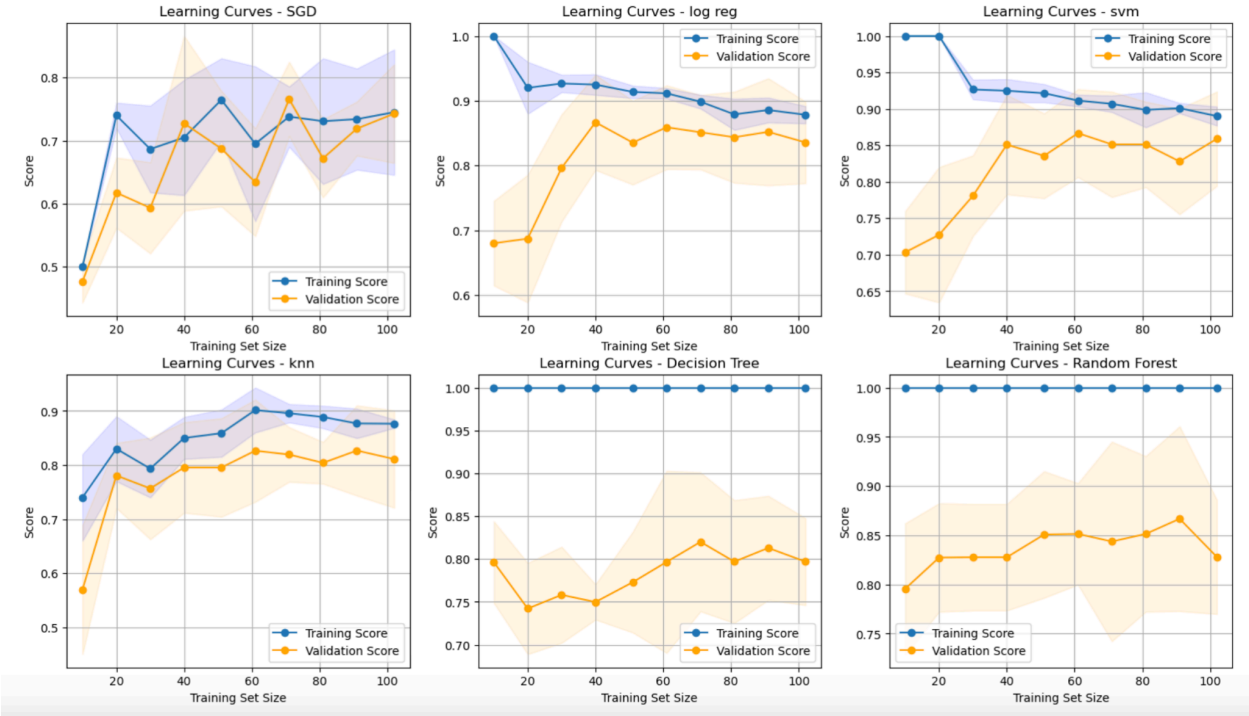
This is a classification task which will classify dance movements into Jazz Ballet or House. The project uses supervised learning, as I have a set of labeled data and am training the model using the provided labels. As far as performance is concerned, this is not a task with devastating effects for an incorrect outcome, so I will be pleased with an accuracy above 80%. However, my goal is an accuracy level at or above 90%. My desired outcome would also include f1-scores at or above 0.90.

To evaluate the performance of my model, I will use multiple metrics. First, because my two classes are balanced by having an equal number of samples, the accuracy score is a good metric to use to look at the overall performance. Another metric I will look at is the F1-score, which is a good tool for overall performance because it balances the importance of both precision and recall. I will also take training time into consideration because we want to have efficient algorithms whenever possible.

My data is coming from a video database called AIST++. The data can be found in my data/ folder in my git repository. The first significant issue I encountered was the usability of this data set. Unfortunately, it is not a database with a large number of dances formatted as samples. Instead, each of the samples are saved as separate pkl files. This gave me the tedious task of loading in all of the samples from each genre of dance individually. However, with the help of the documentation, I was able to easily distinguish genres from the naming conventions and build my own dataframe by loading in each sample (using loops) and combining them into a Pandas dataframe. I believe this data will allow me to achieve my goal because it is quite extensive. Each dance category has multiple choreographers, dancers, and the videos are taken from multiple angles. This leads me to believe that the solution I come up with using this data will be more likely to work with outside videos than that of a less robust dataset.

The different models I am testing are logistic regression, support vector machines, K-nearest neighbors, decision tree classification, and random forest classification. Logistic Regression is a

linear classifier that works well for simple, linearly separable data but struggles with non-linear relationships. Support Vector Machines (SVM) find optimal decision boundaries and handle non-linear data using kernels but require careful parameter tuning and can be computationally intensive. K-Nearest Neighbors (KNN) is a simple, instance-based algorithm that classifies based on the majority class of nearest neighbors. However, it is sensitive to irrelevant features and can be slow during inference. Decision Tree Classification splits data based on feature values to capture non-linear relationships but is prone to overfitting on complex datasets. Random Forest Classification, an ensemble of decision trees, reduces overfitting and improves robustness but is less interpretable and computationally heavier. As one can see, these models all have different strengths and weaknesses so some may perform better than others. We also have to take into account the baseline model. These models will be compared against a baseline of random guessing, which, due to the balanced dataset, provides an accuracy benchmark of 50%.



The first step of training a model is the train-test-split. I split my data set into 80% training data and 20% testing data. I then ran the models on the training sets. I used learning curves (image above) to track the training. Something to look for is a good learning curve should have both training and validation scores increase, converge, and stay close together. The scores should eventually plateau. The KNN, logistic regression, and SVM learning curves are all good fits. The KNN curve fits the description quite well, but plateaus at a slightly lower accuracy. The logistic regression and SVM graphs start at 1, which can be a sign of overfitting, but since the validation score is not low, it seems to just be a sign that the model is fitting perfectly. Overfitting tends to look like a much higher training score than validation score which seems to be the case with the decision tree and random forest graphs. I wouldn't say any of these are underfitting, which is

when both scores remain low and do not improve much. While the scores are low for the SGD graph compared to the rest, they do improve a decent amount, so they do not meet the second criteria.

After training the data, I can see that the SVM and logistic regression models performed the best on accuracy and f1-scores which are the metrics I decided to look most closely at. They both had a baseline accuracy of 0.88 and f1-scores of 0.87 and 0.88. This is pretty impressive for the first attempt. Now, these two do take longer to train than some of the other models, with logistic regression taking about 0.03 seconds and SVM taking 0.045 seconds. SGD, KNN, and the decision tree classification were much shorter, but much less accurate. The random forest classifier took longer, 0.09 seconds, and was less accurate with a score of 0.75. I would say the best algorithm for the task would have to be the logistic regression model because of its high accuracy and f1-scores and its lower time requirement compared to the other very successful model, SVM. Some models performed better than others due to their different strengths and weaknesses. For example, decision trees and KNN tend to overfit, and we definitely saw overfitting for the decision tree model. SVMs excel with complex boundaries. Random forest models need quite a bit of parameter tuning for best performance.

After this analysis, I have determined that the best model for this task is logistic regression after scaling with StandardScaler and using PCA for dimension reduction. The accuracy level of the best model was 0.91 and the f1-scores for ballet-jazz and house are 0.90 and 0.91, respectively. These numbers met my goals of 0.90 for the accuracy level and f1-scores. The close second model is an SVM because it achieved the same scores, but just had a longer computing time. I do still think there are some things that went wrong and could be improved upon with this project given more time. First, a lot of my parameter tuning yielded worse results than the basic support vector classifier. I would like to study that behavior more by looking at the effects with other classifiers, changing the train-test-split, among other adjustments. With more time, I would also like to set my expectations higher and aim to achieve an accuracy level and f1-score of 0.95 or higher. All in all, this assignment was great experience working through a machine learning project from start to finish.

Works Cited:

Shuhei Tsuchida, Satoru Fukayama, Masahiro Hamasaki and Masataka Goto. AIST Dance Video Database: Multi-genre, Multi-dancer, and Multi-camera Database for Dance Information Processing. In Proceedings of the 20th International Society for Music Information Retrieval Conference (ISMIR 2019), 2019.